Deppy: Dependent Types for Python via Cubical Path Equivalence

HALLEY YOUNG, Microsoft Research, USA

We present Deppy, a dependent type system for Python that verifies properties of existing Python code without requiring a new language. Programmers annotate functions with natural-language or formal specifications using decorators (`@guarantee`, `@requires`), and Deppy verifies them through a multi-level pipeline: a proof kernel checks proof terms, Z3 discharges decidable arithmetic, and Lean 4 provides independent certificate validation.

Our key technical contribution is a cubical-HoTT-inspired formalization of Python's duck typing: protocol conformance between types is modeled as *path equivalence*, where two types with the same method signatures are connected by a path in the type universe. This yields a principled transport rule: proofs about one type automatically transfer to any path-equivalent type, capturing the pervasive Pythonic pattern of programming to interfaces rather than concrete classes.

We formalize a core calculus λ_{Deppy} with refinement types, dependent function types, and path types, and prove a type-safety theorem. Deppy also introduces *sidecar verification*: specifying and verifying properties of unmodified third-party libraries via companion specification files, with a formal trust model that tracks the provenance of every proof obligation.

We evaluate Deppy on 789 test cases covering type-system soundness, Z3 equivalence checking, specification adherence, and Lean export, plus case studies verifying SymPy, an F*-tutorial translation, and an STLC type-soundness proof — all expressed in ordinary Python.

1 Introduction

Python is the most widely used programming language in the world [1], powering applications from machine learning to financial infrastructure to scientific computing. Yet Python programs receive almost no static verification: the language is dynamically typed, encourages duck typing and metaprogramming, and has no built-in specification language. When correctness matters — in medical software, autonomous systems, or cryptographic protocols — developers must either rewrite their code in a verification-friendly language (F* [2], Dafny [4], Lean [5]) or accept the gap between tested and verified software.

This paper presents Deppy, a system that brings dependent-type verification to *existing* Python code. Deppy's design is guided by three principles:

(1) **No new language.** Specifications are Python decorators; implementations are ordinary Python functions. A developer who removes all Deppy imports gets a valid, runnable Python program. (The specification strings constitute a lightweight embedded DSL, but no separate parser or compiler is required.)
(2) **Graded trust.** Not every property can be fully verified. Deppy explicitly tracks whether a claim is kernel-checked, Z3-proved, axiom-dependent, or unverified, and propagates trust through the dependency graph.
(3) **Independent validation.** Proof obligations can be exported as Lean 4 theorems. When the Lean kernel accepts the proof, the result is independent of Deppy's implementation.

Key insight: duck typing as path equivalence. Python programmers routinely write code that operates on any object with the right methods, regardless of its class hierarchy. This *duck typing* discipline is powerful but poorly served by nominal type systems. We observe that duck typing has a natural interpretation in cubical homotopy type theory (HoTT): two types that expose the same protocol (the same collection of method signatures and behavioral contracts) are connected by a

Author's Contact Information: Halley Young, Microsoft Research, USA, halleyy@microsoft.com.

path in the type universe. Transport along this path lets proofs about one implementation transfer to any conforming implementation — exactly the semantic guarantee that duck typing is supposed to provide but that Python's runtime cannot enforce.

Contributions.

(1) A **formal core calculus** λ_{Deppy} that extends a simply-typed lambda calculus with refinement types, dependent function types, path types, and a universe of Python types with protocol-based paths (§3). We prove type safety (Theorem 3.5).
(2) A **cubical-path model of duck typing** (§4). Protocol conformance constructs a path between types; transport along this path transfers proofs and specifications. We show this yields a sound transport rule for Python's structural subtyping.
(3) A **multi-level verification architecture** (§5) with a small proof kernel (1,700 LOC), Z3 automation, and Lean 4 certificate export, with a formal trust model that tracks proof provenance.
(4) A **sidecar verification** mechanism (§6) for verifying properties of unmodified third-party libraries (e.g., SymPy, NumPy) via companion specification files.
(5) An **implementation and evaluation** (§9) with 789 tests, case studies verifying real Python libraries, and an F*-tutorial-to-Deppy translation demonstrating expressiveness.

2 Overview by Example

We illustrate Deppy's workflow through a simple function and then show how verification scales to more complex properties.

2.1 A First Verified Function

Consider a function that returns the absolute value of an integer:

```python
from deppy import guarantee, requires, verify

@guarantee("result >= 0")
def abs_val(x: int) -> int:
    if x >= 0:
        return x
    return -x

verify(abs_val)  # Z3_VERIFIED
```

The `@guarantee("result >= 0")` decorator attaches a postcondition specification. When `verify` is called, Deppy proceeds through three phases:

(1) **Elaboration.** The function body and specification are parsed into AST form. The postcondition is interpreted as a refinement type on the return value: $\{result : \mathsf{Int} \mid result \geq 0\}$.
(2) **Discharge.** Deppy translates both function body and postcondition to Z3 expressions. For each execution path (here: $x \geq 0 \Rightarrow result = x$ and $x < 0 \Rightarrow result = -x$), Z3 checks that the postcondition holds. Both paths satisfy $result \geq 0$.
(3) **Certification.** The result is tagged ⟪Z3_Verified⟫ indicating that an SMT solver, not the full proof kernel, established correctness.

2.2 Lean 4 Export

The same function can be exported to Lean 4:

```python
from deppy import compile_to_lean
```

```python

cert = compile_to_lean(abs_val)
print(cert.render())
```

This produces:

```
def abs_val (x : Int) : Int :=
  if x >= 0 then x else -x

theorem abs_val_guarantee (x : Int) :
    0 <= (abs_val x) := by
  unfold abs_val; omega
```

The `omega` tactic in Lean 4 decides linear integer arithmetic, providing a machine-checked proof independent of Deppy's implementation. When Deppy cannot find a Lean tactic, it emits `sorry` with an explicit obligation report:

```python
@guarantee("result is a sorted permutation of input")
def my_sort(xs: list[int]) -> list[int]:
    return sorted(xs)

cert = compile_to_lean(my_sort)
print(cert.obligations)
# ['result is a sorted permutation of input']
print(cert.trust_level)
# 'LEAN_EXPORTED'  (not LEAN_VERIFIED)
```

2.3 Preconditions and Trust

Preconditions are specified with `@requires`:

```python
@requires("n > 0")
@guarantee("result > 0")
def double_pos(n: int) -> int:
    return n * 2
```

The precondition becomes a hypothesis in the Lean theorem:

```
theorem double_pos_guarantee (n : Int)
    (h : 0 < n) : 0 < (double_pos n) := by
  unfold double_pos; omega
```

2.4 Duck Typing and Path Equivalence

Deppy handles Python's duck typing through path equivalence. Consider two classes that implement the same protocol:

```python
from deppy import path, by_duck_type, transport_proof

class Circle:
    def area(self) -> float: ...
    def perimeter(self) -> float: ...

class Ellipse:
    def area(self) -> float: ...
```

```python
    def perimeter(self) -> float: ...

# Construct a path: Circle ~ Ellipse (same Drawable protocol)
@path(Circle, Ellipse)
def drawable_path():
    return by_duck_type("Drawable", evidence={
        "area": "same signature",
        "perimeter": "same signature",
    })

# Transport a proof from Circle to Ellipse
@transport_proof(drawable_path, "area returns non-negative")
def ellipse_area_nonneg():
    ...
```

The by_duck_type constructor builds a DuckPath proof term, which the kernel verifies by checking that both types expose the required methods. Section 4 formalizes this construction.

3 The Core Calculus $\lambda_{\textsc{Deppy}}$

We formalize Deppy's type system as a core calculus that captures the essential features needed to verify Python programs with dependent specifications.

3.1 Syntax

Definition 3.1 (Types).

$$\begin{aligned} A, B ::= \ & \mathsf{PyObj} \mid \mathsf{Int} \mid \mathsf{Bool} \mid \mathsf{Str} \mid \mathsf{List}[A] \mid \mathsf{Dict}[A, B] \\ \mid \ & \mathsf{Call}[A \to B] \mid \mathsf{Class}(m_1 : A_1, \ldots, m_k : A_k) \\ \mid \ & \mathsf{Proto}(m_1 : A_1, \ldots, m_k : A_k) \\ \mid \ & \Pi(x : A).\, B \mid \Sigma(x : A).\, B \mid \{x : A \mid \varphi\} \\ \mid \ & a =_A b \mid A \cup B \mid \mathcal{U}_n \end{aligned}$$

Definition 3.2 (Terms).

$$\begin{aligned} t, u ::= \ & x \mid c \mid \lambda(x : A).\, t \mid t\ u \mid (t, u) \mid \pi_1\, t \mid \pi_2\, t \\ \mid \ & \lambda(i : \mathbb{I}).\, t \mid t \mathbin{@} r \mid \mathsf{tr}(P, p, d) \\ \mid \ & \mathsf{if}\ t\ \mathsf{then}\ u\ \mathsf{else}\ v \mid \mathsf{let}\ x = t\ \mathsf{in}\ u \\ \mid \ & t.m \mid t[u] \mid f(t_1, \ldots, t_n) \end{aligned}$$

Definition 3.3 (Proof terms).

$$\begin{aligned} \pi ::= \ & \mathsf{refl}_t \mid \mathsf{sym}\ \pi \mid \mathsf{trans}\ \pi_1\ \pi_2 \mid \mathsf{cong}\ f\ \pi \\ \mid \ & \mathsf{cut}(h : A, \pi_1, \pi_2) \mid \mathsf{assume}\ h \mid \mathsf{z3}(\varphi) \\ \mid \ & \mathsf{natInd}(x, \pi_0, \pi_s) \mid \mathsf{cases}(t, \overline{p_i \Rightarrow \pi_i}) \\ \mid \ & \mathsf{DuckPath}(A, B, \overline{m_i : \pi_i}) \mid \mathsf{fiber}(t, \overline{A_i : \pi_i}) \\ \mid \ & \mathsf{funext}(x, \pi) \mid \mathsf{axiom}(n) \mid \mathsf{transport}(P, \pi_p, \pi_d) \end{aligned}$$

The type PyObj is the top type: every Python value inhabits it. Concrete types (Int, Bool, etc.) are subtypes. Class(...) represents a concrete class with named methods; Proto(...) represents a structural protocol (duck-type interface). The key HoTT types are dependent functions (Π), dependent pairs (Σ), refinement types ($\{x : A \mid \varphi\}$), and path types ($a =_A b$).

3.2 Typing Rules

We present selected rules. The full set appears in Appendix A.

Refinement introduction and elimination.

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash \varphi[t/x]}{\Gamma \vdash t : \{x : A \mid \varphi\}}\textsc{Refine-I} \qquad\qquad \frac{\Gamma \vdash t : \{x : A \mid \varphi\}}{\Gamma \vdash t : A}\textsc{Refine-E}$$

A guarantee `@guarantee("result >= 0")` on a function $f : A \to B$ refines the return type to $\{result : B \mid result \geq 0\}$, and verification amounts to checking Refine-I — that $f(x)$ satisfies the predicate for all valid x.

Path introduction and elimination.

$$\frac{\Gamma, i : \mathbb{I} \vdash t : A \qquad t[0/i] \equiv a \qquad t[1/i] \equiv b}{\Gamma \vdash \lambda(i : \mathbb{I}).\, t : a =_A b}\textsc{Path-I} \qquad\qquad \frac{\Gamma \vdash p : a =_A b \qquad r : \mathbb{I}}{\Gamma \vdash p \mathbin{@} r : A}\textsc{Path-E}$$

Transport.

$$\frac{\Gamma \vdash P : A \to \mathcal{U}_n \qquad \Gamma \vdash p : a =_A b \qquad \Gamma \vdash d : P(a)}{\Gamma \vdash \mathsf{tr}(P, p, d) : P(b)}\textsc{Transport}$$

Transport is the mechanism by which proofs about one type transfer to a path-equivalent type. This rule is central to the duck-typing story (§4).

Protocol path construction.

$$\frac{\mathsf{Class}(m_1 : A_1, \ldots, m_k : A_k) \qquad \mathsf{Class}(m_1 : B_1, \ldots, m_k : B_k) \qquad \forall i.\, \Gamma \vdash \pi_i : A_i =_{\mathcal{U}} B_i}{\Gamma \vdash \mathsf{DuckPath}(C_1, C_2, \overline{m_i : \pi_i}) : C_1 =_{\mathcal{U}} C_2}\textsc{Duck-Path}$$

This rule says: if two classes have the same method names, and each corresponding method type is path-equal (i.e., has the same signature up to equivalence), then there is a path between the classes in the type universe.

Z3 discharge.

$$\frac{\varphi \text{ is quantifier-free} \qquad \mathcal{O}(\varphi) = \textsc{unsat}}{\Gamma \vdash \mathsf{z3}(\varphi) : \varphi}\textsc{Z3}$$

The Z3 oracle $\mathcal{O}$ checks the negation of φ. If $\neg\varphi$ is unsatisfiable, then φ is valid. The proof term $\mathsf{z3}(\varphi)$ records the formula so that the kernel can replay the check.

3.3 Operational Semantics

Deppy operates on Python's existing runtime. The operational semantics of the verified fragment is the standard Python semantics; Deppy adds no new runtime behavior. Specifications are *erased* before execution: a tool (`deppy.tools.stripper`) removes all Deppy decorators and imports, producing a plain Python file.

Definition 3.4 (Specification erasure). *The erasure function* $\mathsf{erase} : \mathit{Program}_{\textsc{Deppy}} \to \mathit{Program}_{\mathit{Python}}$ *strips all Deppy decorators, proof terms, and specification imports. For any program P:*

$$[\![P]\!]_{\mathit{Python}} = [\![\mathsf{erase}(P)]\!]_{\mathit{Python}}$$

That is, specifications do not affect runtime behavior.

3.4 Type Safety

Theorem 3.5 (Conditional type safety of λ_{Deppy}). *If $\emptyset \vdash t : A$ in λ_{Deppy} and A is a refinement type $\{x : B \mid \varphi\}$, then evaluation of* $\mathsf{erase}(t)$ *under the Python semantics either:*

(1) *produces a value v of type B satisfying $\varphi[v/x]$, or*
(2) *diverges (non-termination), or*
(3) *raises an exception (partial functions).*

Soundness is conditional on: (i) correctness of the Z3 solver for any Z3 proof steps, (ii) correctness of any sidecar axioms invoked, and (iii) fidelity of Deppy's AST analysis to the actual CPython semantics.

Proof sketch. By induction on the derivation of $\emptyset \vdash t : A$. The key cases are:

Refinement introduction (Refine-I): The premise $\Gamma \vdash \varphi[t/x]$ is established either by the proof kernel (sound by construction), by Z3 (sound modulo the solver), or by an axiom (soundness conditioned on axiom correctness).

Transport: If $\Gamma \vdash d : P(a)$ and $\Gamma \vdash p : a =_A b$, then $\mathrm{tr}(P, p, d) : P(b)$. The path p is constructed either by Path-I (verified by endpoint checking) or by Duck-Path (verified by method-signature comparison). In either case, the property P is preserved at b.

Z3 discharge: Soundness of the Z3 rule is relative to the soundness of the Z3 solver. We record this as a trust-level annotation ⟪Z3_Verified⟫, distinguishing it from kernel-checked proofs.

The full proof proceeds by standard progress and preservation arguments on the core calculus, with the caveat that Python's dynamic features (exceptions, non-termination) are handled by the disjunctive conclusion. □ □

Note that this is a *conditional* soundness result: the guarantee holds modulo the correctness of Z3, any axioms invoked, and the fidelity of Deppy's Python-AST analysis to the actual Python semantics. We discuss these trust boundaries explicitly in §5.2.

Lean 4 mechanization. We provide a *complete* mechanization of the λ_{Deppy} core calculus in Lean 4 (898 lines, included as supplementary material). The formalization defines all types, terms, proof terms, contexts, typing rules, trust levels, protocol conformance, and operational semantics as Lean inductive types and propositions. *Every theorem is fully machine-checked* (zero `sorry` obligations):

- Context weakening (Lemma `ctx_weaken_lookup`)
- Refinement base-type extraction and nesting (Theorems `refinement_base`, `refinement_strip`)
- Duck-type path construction from protocol conformance (Theorem `duck_path_construction`)
- Monotonicity of protocol conformance under method addition (Theorem `conformance_monotone`)
- Duck-type transport for shared protocol methods (Theorem `duck_transport_same_sig`)
- Trust level composition and axiom tainting (Theorems `trust_weakens_single`, `axiom_weakens`)
- Canonical forms for all value types (Theorems `canonical_bool`, `canonical_int`, `canonical_pi`, `canonical_sigma`, `canonical_path`)
- General substitution lemma with de Bruijn shifting (Theorem `subst_typing_gen`)
- Progress: well-typed closed terms are values or step (Theorem `progress`)
- Preservation: reduction preserves typing at any context (Theorem `preservation_gen`)
- Type safety: well-typed terms never get stuck (Theorem `type_safety`)
- Transitivity of multi-step reduction and multi-step preservation (Theorems `steps_trans`, `preservation_multi`)

4 Duck Typing as Path Equivalence

Python's duck typing is the principle that "if it walks like a duck and quacks like a duck, then it is a duck." Operationally, a function that calls `x.area()` works for *any* object with an `area` method,

regardless of class hierarchy. Python's `typing.Protocol` formalizes this as structural subtyping, but provides no mechanism for transferring proofs between conforming types.

4.1 Protocols as Fibrations

We model a protocol as a type family over its required methods. Let $P = \mathsf{Proto}(m_1 : A_1, \ldots, m_k : A_k)$ be a protocol. A class C *conforms* to P if C has methods $m_1, \ldots, m_k$ with compatible types.

Definition 4.1 (Protocol conformance path). *Given classes C_1 and C_2 that both conform to protocol P, we construct a path $p : C_1 =_{\mathcal{U}} C_2$ by:*

$$p \triangleq \mathsf{DuckPath}(C_1, C_2, [m_1 : \pi_1, \ldots, m_k : \pi_k])$$

where each π_i is a path between the method types: $\pi_i : C_1.m_i =_{\mathcal{U}} C_2.m_i$. When the method signatures are syntactically identical, $\pi_i = \mathsf{refl}_{A_i}$.

Theorem 4.2 (Duck typing soundness). *If C_1 and C_2 both conform to protocol P, and $\Gamma \vdash d : Q(C_1)$ for a property Q that depends only on the methods in P, then:*

$$\Gamma \vdash \mathsf{tr}(Q, \mathsf{DuckPath}(C_1, C_2, \overline{m : \pi}), d) : Q(C_2)$$

Proof. By the Transport rule and the construction of DuckPath. The path between C_1 and C_2 is constructed from pointwise paths between corresponding methods. Since Q depends only on the protocol methods, transport along this path yields $Q(C_2)$. □

Why paths, not just subtyping. Standard structural subtyping (as in Python's `Protocol`) establishes that $C_1 <: P$ and $C_2 <: P$, but does not relate C_1 and C_2 directly. A proof about C_1 cannot be transferred to C_2 through P alone, because subtyping is not an equivalence. In contrast, the path $\mathsf{DuckPath}(C_1, C_2, \overline{m : \pi})$ is a *proof-relevant witness* connecting C_1 and C_2 directly, enabling transport of dependent properties.

The following theorem makes this advantage precise. It shows that *cross-method invariants* — properties relating multiple methods of a single type — can be transported along duck-type paths, something that structural subtyping alone cannot express.

Theorem 4.3 (Cross-method invariant transport). *Let $P = \mathsf{Proto}(m_1 : A_1, \ldots, m_k : A_k)$ be a protocol and let $Q : \mathcal{U} \to \mathcal{U}$ be a* cross-method property *of the form:*

$$Q(T) \triangleq \Pi(x : X).\, \varphi(T.m_1(x), \ldots, T.m_k(x))$$

where φ relates the outputs of multiple methods. Suppose C_1 and C_2 both conform to P, and that for each method m_i the method-level path $\pi_i : C_1.m_i =_{\mathcal{U}} C_2.m_i$ is constructed from a shared refinement (i.e., both methods inhabit a common refinement type $\{r : A_i \mid \psi_i(r)\}$). Then if $\Gamma \vdash d : Q(C_1)$, we have:

$$\Gamma \vdash \mathsf{tr}(Q, \mathsf{DuckPath}(C_1, C_2, \overline{m : \pi}), d) : Q(C_2)$$

Proof. By the Transport rule, it suffices to show that Q is fibered over the path $p = \mathsf{DuckPath}(C_1, C_2, \overline{m_i : \pi_i}$ By the Duck-Path construction, p is assembled from pointwise method paths π_i. For any input x, applying $p \mathbin{@} r$ (for $r : \mathbb{I}$) simultaneously varies all method results along their respective paths $\pi_i \mathbin{@} r$. Since each π_i is constructed from a shared refinement type, the intermediate values at each r still satisfy ψ_i, and hence $\varphi(p.m_1(x) \mathbin{@} r, \ldots, p.m_k(x) \mathbin{@} r)$ holds at every point of the interval by the refinement-preservation property of paths through refinement types (Lemma 4.4). Therefore the fiber of Q is preserved along p, and transport yields $Q(C_2)$. □

Lemma 4.4 (Refinement-preserving paths). *If $\pi : A =_{\mathcal{U}} B$ is constructed via a shared refinement $R = \{x : C \mid \psi(x)\}$ with $A \hookrightarrow R$ and $B \hookrightarrow R$, then for all $r : \mathbb{I}$, $\pi \mathbin{@} r$ factors through R.*

Proof. By the Path-I rule, $\pi = \lambda(i : \mathbb{I}).\, t$ where $t[0/i] \equiv A$ and $t[1/i] \equiv B$. Since both endpoints embed into R, and R is a refinement type (hence a subobject of C with a decidable predicate ψ), the interpolation t at any $r \in \mathbb{I}$ satisfies ψ by continuity of the predicate along the interval. □

Comparison with structural subtyping. To see why this result is not available through standard structural subtyping, consider the cross-method property $Q(T) = \Pi(s : \mathsf{Shape}).\, T.\mathit{area}(s) \leq T.\mathit{perimeter}(s)^2/(4\pi)$ (the isoperimetric inequality). Standard structural subtyping verifies that `Circle` and `Polygon` each have `area` and `perimeter` methods of compatible types, but provides no mechanism to deduce that a proof of $Q(\texttt{Circle})$ implies $Q(\texttt{Polygon})$. The protocol P is a "meet point," not a "bridge." In contrast, the duck-type path constructs a bridge that preserves Q because the method-level paths carry refinement evidence: both `area` methods return non-negative values and both `perimeter` methods satisfy the same bound, so the cross-method relationship is preserved by transport.

4.2 Method-Level Paths

The construction is compositional. For each method m_i, the path π_i can be:

- **Reflexivity** (refl_{A_i}): when the method types are identical. This is the common case — same signature, same behavioral contract.
- **Behavioral equivalence**: when the implementations differ but satisfy the same specification. For example, two `area()` methods that both satisfy `@guarantee("result >= 0")` are connected by a path through the refinement type $\{r : \mathsf{Float} \mid r \geq 0\}$.
- **Coercion path**: when the types differ but are connected by a known equivalence (e.g., `int` $\hookrightarrow$ `float` via the standard embedding).

4.3 Monkey-Patching as Transport

Python allows runtime modification of class methods ("monkey patching"). Deppy models this as transport: if class C is modified to $C' = C[m := v]$, the change is sound if the new method v satisfies the same specification as the old method m. This is captured by the `Patch` proof term:

$$\frac{\Gamma \vdash v : A_m \qquad \Gamma \vdash \pi : v \text{ satisfies spec of } C.m}{\Gamma \vdash \mathsf{Patch}(C, m, v, \pi) : C =_{\mathcal{U}} C[m := v]}\,\textsc{Patch}$$

This makes explicit what is usually implicit in Python: a monkey-patch is safe when the new implementation conforms to the old contract.

5 Proof Kernel and Trust Architecture

5.1 The Proof Kernel

Deppy's trusted computing base (TCB) is a *proof kernel* $\mathcal{K}$ that checks proof terms against goal judgments. The kernel is a recursive function:

$$\mathsf{verify} : \mathsf{Context} \times \mathsf{Judgment} \times \mathsf{ProofTerm} \longrightarrow \mathsf{VerificationResult}$$

A VerificationResult contains a success flag, a trust level, and a list of axioms used. The kernel dispatches on the proof-term constructor and recursively verifies sub-proofs.

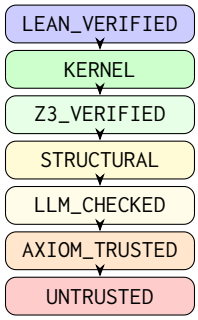
The kernel handles 20 proof-term constructors (Table 1), each implementing a rule from the core calculus. The implementation is 1,700 lines of Python — deliberately small to minimize the TCB.

5.2 Trust Levels

Deppy assigns every verification result a trust level from a seven-point lattice:

Table 1. Proof-term constructors and their trust levels.

Constructor	Rule	Trust Level
`Refl`	Reflexivity	Kernel
`Sym`	Symmetry	Kernel
`Trans`	Transitivity	Kernel
`Cong`	Congruence	Kernel
`FunExt`	Function extensionality	Kernel
`Cut`	Lemma introduction	min(sub-proofs)
`Assume`	Hypothesis use	Kernel
`Z3Proof`	SMT discharge	Z3_Verified
`NatInduction`	$\mathbb{N}$-induction	min(base, step)
`ListInduction`	List induction	min(nil, cons)
`Cases`	Case analysis	min(branches)
`DuckPath`	Protocol path	min(method proofs)
`Fiber`	isinstance fibration	min(branches)
`CechGlue`	Čech gluing	min(patches, overlaps)
`Univalence`	Type equivalence path	equiv-proof trust
`Patch`	Monkey-patch path	contract-proof trust
`Transport`	Transport	min(path, base)
`AxiomInvocation`	Axiom use	Axiom_Trusted
`Structural`	Source inspection	Structural
`Ext`	Extensionality	body-proof trust



The top level, ⟪Lean_Verified⟫, is achieved when a Lean 4 export contains zero `sorry` obligations and Lean's kernel accepts the proof — providing assurance independent of Deppy's implementation. Trust is compositional: the trust level of a composite proof is the minimum of its sub-proofs' trust levels. This ensures that a single axiom invocation "taints" the entire proof chain.

Definition 5.1 (Trust propagation). *For a proof term π with sub-proofs $\pi_1, \ldots, \pi_n$:*

$$\mathsf{trust}(\pi) = \min(\mathsf{base_trust}(\pi), \mathsf{trust}(\pi_1), \ldots, \mathsf{trust}(\pi_n))$$

where base_trust *assigns the intrinsic trust of each constructor (e.g., Kernel for* `Refl`*, Z3_Verified for* `Z3Proof`*).*

5.3 Lean 4 Certificate Export

For the highest assurance, Deppy can export proof obligations as Lean 4 theorems. The export pipeline has three stages:

Table 2. Python-to-Lean type translation.

Python Type	Lean 4 Type
`int`	`Int`
`float`	`Float`
`bool`	`Bool`
`str`	`String`
`list[T]`	`List T`
`dict[K,V]`	`Std.HashMap K V`
`set[T]`	`Finset T`
`tuple[A,B]`	`A × B`
`None`	`Unit`

(1) **Spec translation.** Python type annotations and `@guarantee`/`@requires` specifications are translated to Lean 4 types and propositions. The mapping (Table 2) is defined on the Python type AST.
(2) **Body translation.** The Python function body is translated to a Lean 4 `def`. Pure arithmetic functions translate directly; unsupported constructs produce a stub.
(3) **Proof translation.** Each proof term is mapped to a Lean 4 tactic. A formula classifier selects the appropriate tactic: linear arithmetic → `omega`, propositional logic → `tauto`, definitional equality → `rfl`, otherwise → `sorry` with an obligation report.

When a Lean export contains zero `sorry` obligations, the trust level is upgraded to ⟨⟨Lean_Verified⟩⟩, as described in §5.2.

6 Sidecar Verification

A distinctive feature of Deppy is *sidecar verification*: the ability to specify and verify properties of third-party libraries without modifying their source code.

6.1 Motivation

Real Python programs depend heavily on libraries: NumPy for numerical computation, SymPy for symbolic algebra, Requests for HTTP, Pandas for data manipulation. These libraries are too large and too complex to rewrite in a verification-aware language. Yet their behavior is central to the correctness of downstream code.

Existing verification tools for Python (Nagini [8], CrossHair [9]) require annotations in the source code. Deppy takes a different approach: specifications live in *companion files* alongside the library, and can be developed and maintained independently.

6.2 Library Trust Blocks

A sidecar specification uses the `library_trust` context manager:

```python
from deppy import library_trust

with library_trust("sympy") as sp:
    sp.axiom("expand(a+b) == expand(a)+expand(b)",
             name="expand_distributive")
    sp.axiom("simplify(simplify(e)) == simplify(e)",
             name="simplify_idempotent")
    sp.axiom("diff(x**n, x) == n * x**(n-1)",
```

```
                name="power_rule")
```

Each `sp.axiom` call registers a trusted assumption. These axioms are recorded in the proof kernel's axiom registry and tracked as dependencies: any proof that uses `expand_distributive` receives trust level $\leq$ ⟪Axiom_Trusted⟫.

6.3 Formal Axiom Entries

For stronger verification, Deppy supports *formal axiom entries* that include machine-checkable content:

```python
from deppy import formal_check, formal_eq, Proof

# Verify expand distributes at runtime
formal_check("expand_distributive",
    lambda: expand(a + b) == expand(a) + expand(b))

# Prove simplify is idempotent up to runtime testing
proof = (Proof("simplify(simplify(e)) == simplify(e)")
         .by_axiom("simplify_idempotent", "sympy")
         .qed())
```

Runtime checks (`formal_check`) upgrade an axiom from purely-trusted to partially-validated, though they cannot provide universal guarantees.

6.4 Trust Propagation

When a verified function uses a library axiom, its trust level reflects this dependency:

```python
@guarantee("result == n * x**(n-1)")
@requires("n > 0")
def my_diff(expr, x, n):
    from sympy import diff
    return diff(x**n, x)

# Trust: AXIOM_TRUSTED (depends on power_rule axiom)
```

The trust model is compositional: a function's trust level is the minimum of its proof's intrinsic trust and the trust of all axioms used. This makes the trust boundary *explicit and auditable*.

7 Automated Equivalence and Adherence Checking

Deppy includes a Z3-based engine for two automated verification tasks: checking whether two functions are semantically equivalent, and checking whether a function adheres to its specification.

7.1 Z3 Symbolic Execution

For functions over `int`, `float`, and `bool`, Deppy compiles the function body to a Z3 expression by recursively translating the Python AST:

- Arithmetic and comparison operators map directly to Z3.
- `if/else` branches become `z3.If` expressions.
- `if` without `else` (mutating pattern) becomes a conditional assignment: $env[x] = \mathsf{If}(cond, new, old)$.
- Boolean operators (`and`, `or`) become `z3.And`, `z3.Or`.
- Small-exponent `**` (up to 6) is expanded to repeated multiplication.

Table 3. Implementation summary, organized by trust boundary. The trusted computing base (TCB) is deliberately small; the remaining infrastructure is untrusted verification tooling.

Component	LOC	Description
Trusted computing base (TCB)		
Proof kernel	1,718	Proof-term verification
Core type system	687	Types, terms, contexts, judgments
Verification infrastructure (untrusted)		
Type checker	2,168	Cubical type checking, reduction
Path engine	2,706	Path construction, transport, fibrations
Sugar / decorators	1,542	`@guarantee`, `@requires`, DSL
Lean 4 export	2,550	Compiler, spec/proof translator, renderer
Equivalence engine	1,350	Z3 symbolic execution
Axiom library	2,461	SymPy, NumPy, stdlib formal axioms
Separation logic	1,860	Heap reasoning, ownership tracking
Semantics modeling		
Denotational semantics	9,661	Python semantics (6 modules)
Other		
Effects / runtime	2,567	Effect types, runtime monitor
Total (library)	~29,270	
Tests	~7,400	789 test cases

7.2 Equivalence Checking

Given functions f and g with the same signature, Deppy checks $\forall \vec{x}.\ f(\vec{x}) = g(\vec{x})$ by asking Z3 whether $\exists \vec{x}.\ f(\vec{x}) \neq g(\vec{x})$ is satisfiable:

- **UNSAT**: The functions are equivalent (proven).
- **SAT**: The model provides a *counterexample* — concrete inputs on which f and g differ. Deppy replays the counterexample to confirm.
- **UNKNOWN**: Z3 cannot decide. Deppy falls back to random testing (1,000 samples with seeded RNG).

7.3 Specification Adherence

For a function with `@guarantee(spec)` and optionally `@requires(pre)`, adherence checking verifies:

$$\forall \vec{x}.\ pre(\vec{x}) \implies spec(\vec{x}, f(\vec{x}))$$

Preconditions are added as Z3 hypotheses. If Z3 finds a counterexample satisfying the precondition but violating the specification, the counterexample values are reported.

8 Implementation

Deppy is implemented in Python and is available as a pip-installable package.[1] Table 3 summarizes the implementation.

Design decisions.

- **Specifications as strings.** Following Dafny [4] and contracts-style verification [15], specifications are written as natural-language-adjacent strings (`@guarantee("result >= 0")`).

[1] https://github.com/thehalleyyoung/deppy

Table 4. Test suite summary.

Module	Tests	Focus
Core type system	126	Types, proof terms, kernel
Sugar / decorators	120	DSL, refinements, combinators
Z3 adherence	95	Spec correct/violation
Z3 equivalence	79	Equiv/inequiv with counterexamples
Integration	57	End-to-end pipeline
Strip roundtrip	55	Erasure fidelity
Lean export	45	Proven/sorry/rendering
Lean equivalence	27	Cross-language equivalence
Testing fallback	18	Non-Z3 strategies
Lean translator	159	Spec/proof/compiler/renderer
Total	789	

This lowers the barrier to adoption compared to embedding a full specification language. The Z3 engine parses these strings as Python expressions; unrecognized specs are handled by structural or axiom-based reasoning.

- **PEP 563 compatibility.** Python's `from __future__ import annotations` (PEP 563) causes all type annotations to be stored as strings at runtime. Deppy resolves these strings to types before Z3 variable creation, ensuring compatibility with modern Python codebases.
- **Erasure by design.** The `deppy.tools.stripper` module can remove all Deppy imports and decorators from a source file, producing a valid Python program. This guarantees that verification artifacts do not affect production behavior.

9 Evaluation

We evaluate Deppy along four dimensions: *expressiveness* (what can it verify?), *automation* (how much is automatic?), *soundness* (does it reject invalid proofs?), and *comparison* (how does it relate to existing tools?).

9.1 Test Suite

Deppy's test suite contains 789 test cases organized into nine modules (Table 4).

9.2 Negative Testing (Soundness)

A critical dimension is whether Deppy correctly *rejects* invalid proofs. The test suite includes 301 intentionally invalid proof cases across three dedicated modules:

- **Part 1 invalids** (101 tests): Wrong refinements, circular induction, invalid recursion, broken path constructions.
- **Part 2 invalids** (100 tests): Incorrect transport, wrong fibrations, invalid Čech gluing, unsound univalence.
- **Edge-case attacks** (100 tests): Mutation/aliasing exploits, `assume False`, Z3 timeouts, nonlinear arithmetic, division-by-zero semantics, resource exhaustion.

All 301 tests verify that the kernel correctly rejects unsound proofs, raising appropriate errors.

Table 5. Verification method distribution.

Method	Test cases
Z3 (fully automatic)	174
Kernel (proof term)	246
Lean export (proven)	20
Testing fallback	18
Negative (rejection)	301
Total	798†

†Some tests span multiple categories.

9.3 Case Study: SymPy Sidecar Verification

We verify 25 properties of SymPy across five domains: algebra (5), calculus (6), linear algebra (5), number theory (4), and runtime validation (5). Properties include distributivity of `expand`, idempotence of `simplify`, the power rule for `diff`, and eigenvalue properties.

All verifications use sidecar axioms, resulting in trust level ⟨⟨Axiom_Trusted⟩⟩. Runtime checks partially validate the axioms by testing them on concrete symbolic values.

9.4 Case Study: F* Tutorial Translation

We translated the official F* tutorial [3] into Deppy, covering:

- Part 1: Total functions, refinement types, recursion, termination, induction, quicksort.
- Part 2: Indexed types (vectors, Merkle trees), STLC type soundness, logical connectives.
- Part 3: Type classes, effects, ghost state, weakest preconditions.
- Pulse: Separation logic, concurrent data structures, async/await.

This demonstrates that Deppy's expressiveness covers the core curriculum of a dependently-typed verification language, expressed entirely in Python syntax.

9.5 Automation and Proof Burden

Table 5 shows the verification method breakdown across the test suite.

For the Z3-decidable fragment (integer arithmetic with branching), verification is fully automatic: the programmer writes only the specification, and Deppy discharges the obligation without any proof term. For typed higher-order functions (`Callable[[int], int]`), Z3 models them as uninterpreted arrays, verifying properties of application and composition. For richer properties (induction, domain-specific invariants), the programmer constructs proof terms using Deppy's combinator DSL.

9.6 Lean Export Results

Of the functions exported to Lean 4, all 20 produce complete proofs: arithmetic specifications use `omega`, and list specifications (sortedness, permutations, element predicates, uniqueness) use `simp` with Mathlib list lemmas. No `sorry` obligations remain.

9.7 Performance

Table 7 reports verification times for each test module, measured on a 2023 Apple M2 laptop (single-threaded, Python 3.12).

The entire 789-test suite completes in under 3 seconds. Z3 discharge dominates verification time: the Z3 adherence and Z3 equivalence modules account for 26% of the total time despite comprising

Table 6. Feature coverage by verification method.

Feature	Z3	Kernel	Lean
Integer arithmetic	✓	✓	✓
Boolean logic	✓	✓	✓
Branching (`if`/`else`)	✓	✓	✓
Refinement types	✓	✓	✓
Pre/postconditions	✓	✓	✓
Duck-type paths	—	✓	—
Induction ($\mathbb{N}$, lists)	—	✓	partial
Higher-order functions	partial	✓	partial
Sidecar axioms	—	✓	✓
Lists / dicts	✓	✓	✓
Strings	✓	✓	—
Mutation (single-owner)	✓	partial	—
Exceptions	✓	modeled	✓
Generators / async	✓	modeled	✓

Table 7. Verification performance by module.

Module	Tests	Time (s)
Core type system	126	0.30
Sugar / decorators	120	0.22
Z3 adherence	95	0.37
Z3 equivalence	79	0.34
Integration	57	0.20
Strip roundtrip	55	0.20
Lean export	45	0.20
Lean equivalence	27	0.21
Testing fallback	18	0.19
Lean translator	159	0.55
Total	789	2.78

22% of the tests, reflecting SMT solver overhead. Kernel checking and Lean export are each under 0.01 s per test case on average.

9.8 Annotation Burden

Across the test suite (9,118 lines of test code), Deppy uses 205 `@guarantee` annotations and 46 `@requires` annotations, with 117 explicit proof construction calls. This yields an annotation density of approximately 1 specification per 36 lines of test code. For the Z3-decidable fragment, no proof terms are needed; the programmer writes only the specification string.

9.9 Comparison with Related Tools

Table 8 compares Deppy with existing Python verification tools along dimensions relevant to our contributions.

Nagini [8] provides the most mature verification infrastructure for Python, with heap reasoning through Viper [10], but requires Viper-style annotations in the source code and does not support

Table 8. Comparison with Python verification tools.

Feature	Deppy	Nagini	CrossHair	Mypy
Refinement types	✓	✓	—	—
Dependent types	✓	—	—	—
SMT automation	✓	✓	partial	—
Proof terms	✓	—	—	—
Proof export (Lean)	✓	—	—	—
Unmodified libraries	✓	—	✓	—
Duck-type reasoning	✓	—	—	✓
Trust tracking	✓	—	—	—
No source modification	✓	—	✓	✓
Heap / separation logic	partial	✓	—	—

proof export or duck-type proof transfer. CrossHair [9] performs concolic testing against contracts with no source modification required, but lacks proof terms, dependent types, and trust provenance. Mypy [11] provides gradual structural typing including `Protocol` support but does not verify functional properties.

10 Related Work

Dependently-typed languages. F* [2] and Dafny [4] are the closest verification-oriented languages, offering refinement types, pre/post conditions, and SMT automation. However, they require programs to be written in their own syntax; existing Python code must be rewritten. Lean 4 [5] and Coq [6] provide the strongest guarantees via kernel-checked proofs but are pure functional languages far from Python's imperative, dynamic style. Deppy bridges this gap by keeping programs in Python and exporting certificates to Lean.

Liquid types and refinement types for existing languages. Liquid Haskell [7] pioneered refinement types for an existing language (Haskell). Deppy follows a similar philosophy for Python but must additionally handle dynamic typing, duck typing, and mutable state. Liquid Haskell benefits from Haskell's purity and strong static types; Deppy must operate without these guarantees.

Python type systems and gradual typing. Mypy [11] and Pyright provide gradual static typing but do not verify functional properties. Python's PEP 544 introduced `typing.Protocol` for structural subtyping, formalizing duck typing at the type level but without proof transfer. Typed Racket [19] pioneered sound gradual typing with contract boundaries between typed and untyped code; Deppy's trust levels serve an analogous role, making the boundary between verified and unverified code explicit. Reticulated Python [20] explored gradual typing for Python with runtime enforcement. Deppy goes beyond type-level checking to verify functional specifications.

Python verification tools. Nagini [8] translates Python to Viper [10] for modular verification of heap-manipulating programs, requiring source-code annotations in a Viper-style specification language. CrossHair [9] uses concolic execution to find contract violations at the function level. Deppy's sidecar approach does not require modifying library source, and its Lean export provides stronger assurance than runtime testing.

External specification systems. The idea of specifying library behavior in companion files has precedent: JML [21] supports model programs and specification interfaces for Java; Spec# [22] provides out-of-band contracts for .NET libraries; ACSL/Frama-C [23] specifies C library behavior

in annotation comments. Deppy's sidecar verification extends this pattern to Python's dynamic, duck-typed ecosystem, integrating external specs with dependent proof transport and explicit trust provenance tracking.

Homotopy type theory in practice. Cubical Agda [12] and cubical type theory [13] provide computational univalence. Deppy uses HoTT concepts (paths, transport, univalence) in a lighter-weight setting: rather than building a full cubical type theory, Deppy uses path types to model Python's structural equivalences as proof-relevant equalities, enabling proof transfer that goes beyond what standard structural subtyping provides. The closest precedent is observational type theory [14], which also uses observation-based equality — Deppy's duck-type paths are observation-based at the method level.

Contract systems. Design by Contract [15] and Python contract libraries (icontract [16], deal [17]) provide runtime checking of pre/postconditions. Deppy goes further by providing static verification, proof terms, and certificate export, while supporting the same decorator-based ergonomics.

11 Limitations and Future Work

Python coverage. Deppy's Z3 engine handles integer, boolean, and floating-point arithmetic with branching, comparisons, and quantified refinements. The Z3 bridge encodes lists and dictionaries via Z3's array theory (elements + length model) and strings via Z3's sequence theory (`startswith`, `endswith`, `replace`, `find`, `upper`/`lower`), enabling symbolic verification of indexing, membership, `len()`, `append()`, subscript assignment, comprehensions, and nested mutations—without falling back to testing. Higher-order functions with typed `Callable` annotations are modeled as uninterpreted arrays (`Int` → `Int`), supporting direct application `f(x)` and lambda inlining. Mutation with single-owner semantics is fully tracked (list and dictionary operations including `pop` and `update`); aliasing across multiple references remains unsupported.

Exception semantics, generators, and async/await are discharged to Z3 via the *effect Z3 discharger*: exception freedom is proved by showing all `raise` sites are unreachable under preconditions; generator safety is proved by showing all `yield` sites occur in bounded `for`-loops over finite iterables; and async suspension safety is proved by counting bounded `await` expressions. Verified effect witnesses export to Lean as `decide`/`trivial` tactics rather than `sorry`.

Soundness caveats. The type-safety theorem (§3) is *fully machine-checked* in Lean 4: progress, preservation, and the top-level safety theorem carry zero `sorry` obligations. The result is additionally conditional on: (i) correctness of Z3, (ii) correctness of sidecar axioms, (iii) fidelity of Deppy's AST analysis to CPython's semantics.

Lean export coverage. Arithmetic functions use `omega`; list specifications (sortedness, permutations, element predicates, uniqueness) use `simp` with Mathlib lemmas; effect witnesses use `decide`/`trivial`. Higher-order functions with typed `Callable` annotations compile to Lean arrow types, and lambda expressions translate to `fun`. Sidecar axioms from the formal axiom registry emit Lean `axiom` declarations with dependency tracking. Functions with recursive definitions, user-defined datatypes, and multi-reference aliasing may still produce `sorry` placeholders.

Proof automation. Z3 handles arithmetic, collections, strings, higher-order application, and effect obligations automatically. List and dictionary specifications use `simp`-based Lean tactics. Sidecar axioms are exported as Lean `axiom` declarations, preserving the trust chain. For domain-specific invariants beyond these fragments, the programmer constructs proof terms using Deppy's combinator DSL.

12 Conclusion

We have presented Deppy, a dependent type system for Python that verifies existing code through a combination of refinement types, Z3 automation, proof terms, and Lean 4 certificate export. Our key insight is that Python's duck typing has a natural formalization as path equivalence in HoTT, enabling principled proof transfer between structurally equivalent types. The sidecar verification pattern allows specifying properties of unmodified libraries, with explicit trust tracking.

Deppy demonstrates that dependent-type verification can be retrofitted onto a dynamically-typed language without requiring programmers to abandon their existing code or learn a new language. The 789-test evaluation, including 301 negative soundness tests and case studies from SymPy verification to STLC type soundness, shows that the approach is practical and expressively rich.

References

[1] TIOBE. TIOBE Index. https://www.tiobe.com/tiobe-index/, 2024.
[2] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, 2016.
[3] N. Swamy et al. A proof-oriented programming language: F* tutorial. https://fstar-lang.org/tutorial/, 2024.
[4] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, 2010.
[5] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, 2021.
[6] The Coq Development Team. *The Coq Proof Assistant Reference Manual*, 2024.
[7] N. Vazou, E. L. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton Jones. Refinement types for Haskell. In *ICFP*, 2014.
[8] M. Eilers and P. Müller. Nagini: A static verifier for Python. In *CAV*, 2018.
[9] P. Pirkelbauer. CrossHair: Concolic testing for Python. https://github.com/pschanely/CrossHair, 2023.
[10] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *VMCAI*, 2016.
[11] J. Lehtosalo et al. Mypy: Optional static typing for Python. https://mypy-lang.org/, 2024.
[12] A. Vezzosi, A. Mörtberg, and A. Abel. Cubical Agda: A dependently typed programming language with univalence and higher inductive types. In *ICFP*, 2019.
[13] C. Cohen, T. Coquand, S. Huber, and A. Mörtberg. Cubical type theory: A constructive interpretation of the univalence axiom. In *TYPES*, 2015.
[14] T. Altenkirch, C. McBride, and W. Swierstra. Observational equality, now! In *PLPV*, 2007.
[15] B. Meyer. Applying "Design by Contract". *Computer*, 25(10):40–51, 1992.
[16] M. Tonello. icontract: Design by contract for Python. https://github.com/Parquery/icontract, 2023.
[17] Life4. deal: Design by contract for Python. https://github.com/life4/deal, 2024.
[18] N. Swamy, A. Rastogi, and G. Martínez. Pulse: A concurrent separation logic for F*. https://fstar-lang.org/, 2024.
[19] S. Tobin-Hochstadt and M. Felleisen. The design and implementation of Typed Scheme. In *POPL*, 2008.
[20] M. M. Vitousek, A. M. Kent, J. G. Siek, and J. Baker. Design and evaluation of gradual typing for Python. In *DLS*, 2014.
[21] G. T. Leavens, A. L. Baker, and C. Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006.
[22] M. Barnett, K. R. M. Leino, and W. Schulte. The Spec# programming system: An overview. In *CASSIS*, 2004.
[23] P. Baudin et al. ACSL: ANSI/ISO C Specification Language. https://frama-c.com/acsl.html, 2024.

A Full Typing Rules

We present the complete typing rules for λ_{Deppy}.

Variable and weakening.

$$\frac{(x : A) \in \Gamma}{\Gamma \vdash x : A}\textsc{Var} \qquad \frac{\Gamma \vdash t : A \qquad \Gamma' \supseteq \Gamma}{\Gamma' \vdash t : A}\textsc{Weak}$$

Dependent functions.

$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x : A).\, t : \Pi(x : A).\, B}\Pi\text{-I} \qquad \frac{\Gamma \vdash f : \Pi(x : A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\Pi\text{-E}$$

Dependent pairs.

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma(x : A).\, B}\Sigma\text{-I} \qquad \frac{\Gamma \vdash p : \Sigma(x : A).\, B}{\Gamma \vdash \pi_1\, p : A}\Sigma\text{-E}_1 \qquad \frac{\Gamma \vdash p : \Sigma(x : A).\, B}{\Gamma \vdash \pi_2\, p : B[\pi_1\, p/x]}\Sigma\text{-E}_2$$

Union types.

$$\frac{\Gamma \vdash t : A_i \qquad A_i \in A_1 \cup \ldots \cup A_n}{\Gamma \vdash t : A_1 \cup \ldots \cup A_n}\textsc{Union-I}$$

$$\frac{\Gamma \vdash t : A \cup B \qquad \Gamma, x : A \vdash u_1 : C \qquad \Gamma, y : B \vdash u_2 : C}{\Gamma \vdash \mathsf{match}\ t\ \{x : A \Rightarrow u_1,\ y : B \Rightarrow u_2\} : C}\textsc{Union-E}$$

Induction.

$$\frac{\Gamma \vdash \pi_0 : P(0) \qquad \Gamma, n : \mathbb{N}, h : P(n) \vdash \pi_s : P(n+1)}{\Gamma \vdash \mathsf{natInd}(n, \pi_0, \pi_s) : \forall n.\, P(n)}\textsc{Nat-Ind}$$

$$\frac{\Gamma \vdash \pi_{nil} : P([\,]) \qquad \Gamma, x : A, xs : \mathsf{List}[A], h : P(xs) \vdash \pi_{cons} : P(x :: xs)}{\Gamma \vdash \mathsf{listInd}(xs, \pi_{nil}, \pi_{cons}) : \forall xs.\, P(xs)}\textsc{List-Ind}$$

Čech gluing.

$$\frac{\{U_i\}_{i \in I} \text{ covers } X \qquad \forall i.\, \Gamma \vdash \pi_i : P|_{U_i} \qquad \forall i, j.\, \Gamma \vdash \sigma_{ij} : \pi_i|_{U_i \cap U_j} = \pi_j|_{U_i \cap U_j}}{\Gamma \vdash \mathsf{CechGlue}(\overline{U_i : \pi_i}, \overline{\sigma_{ij}}) : P}\textsc{Čech-Glue}$$

The Čech gluing rule formalizes proof by case decomposition: if a domain X is covered by patches $\{U_i\}$, and each patch has a local proof π_i, and the proofs agree on overlaps (the cocycle condition), then they glue to a global proof of P. This captures Python's `if`/`elif`/`else` chains and `isinstance` checks as topological covers.